



MANUALE TECNICO

Laboratorio Interdisciplinare B — a.a. 2025/2026
Università degli Studi dell'Insubria

Autori:
Karakus Yasemin — matricola 762746 — sede VA
Choudhry Maha Ilyas — matricola 747119 — sede VA

Indice

1. Introduzione e obiettivi del progetto
 2. Requisiti, tecnologie e dipendenze
 3. Architettura generale client/server
 4. Struttura del repository e dei package
 5. Protocollo di comunicazione Richiesta/Risposta
 6. Modello di dominio e classi comuni
 7. Progettazione del database e normalizzazione
 8. Modulo serverCM: ServerCM, GestoreClient e service layer
 9. Modulo clientCM: applicazione JavaFX e scene principali
 10. Gestione delle prenotazioni, posti disponibili e concorrenza
 11. Sicurezza, autorizzazione e gestione degli errori
 12. Descrizione completa delle funzioni/metodi principali
 13. Operazioni del protocollo applicativo
 14. Build, esecuzione e distribuzione
 15. Collaudo funzionale e casi di test
 16. Complessità, prestazioni e scelte progettuali
 17. Limiti, assunzioni e possibili estensioni
- Appendice A. Comandi utili
- Appendice B. Riferimenti interni al progetto

1. Introduzione e obiettivi del progetto

CineMax è una piattaforma client/server sviluppata per la gestione di un cinema monosala da 200 posti. L'applicazione permette a diverse categorie di utenti di interagire con lo stesso sistema informativo: il guest può consultare le proiezioni, il cliente può prenotare posti, il proiezionista può gestire il palinsesto e il bigliettaio può consultare le prenotazioni. Il documento descrive le scelte tecniche, la struttura del codice, la progettazione del database e i meccanismi usati per garantire correttezza, sicurezza e gestione della concorrenza.

Il sistema è stato progettato separando l'interfaccia grafica, la logica di comunicazione, la logica applicativa e il database. Questa separazione rende più semplice mantenere il codice, testare i singoli componenti e spiegare il funzionamento durante la presentazione del progetto.

Obiettivo	Soluzione tecnica adottata
Gestione cinema monosala	Un database PostgreSQL con tabelle film, proiezione, utenti e prenotazione; capienza sala pari a 200 posti.
Più utenti collegati	Server TCP sulla porta 4444 con un thread dedicato per ogni client connesso.
Interfaccia desktop	Client JavaFX organizzato in scene separate per login, ricerca, menu cliente, proiezionista e bigliettaio.
Persistenza dati	Accesso al database con JDBC e PreparedStatement.
Sicurezza password	Password cifrate nel database tramite pgcrypto e BCrypt.
Correttezza posti	Calcolo dei posti liberi come 200 meno la somma dei posti prenotati per la specifica proiezione.

2. Requisiti, tecnologie e dipendenze

La piattaforma è realizzata in Java e viene distribuita tramite due JAR eseguibili separati: serverCM.jar e clientCM.jar.

Componente	Versione/scelta	Motivazione
Linguaggio	Java 21	Garantisce compatibilità con le API moderne e coerenza tra IDE e build Maven.
GUI	JavaFX 21.0.5	Permette di realizzare una applicazione desktop con stage, scene, tabelle e form.
Build	Apache Maven	Gestisce dipendenze, compilazione, JAR eseguibili e Javadoc.
Database	PostgreSQL, database dbcm	Salva utenti, film, proiezioni e prenotazioni in modo persistente.
Driver DB	PostgreSQL JDBC 42.7.4	Permette la comunicazione Java-PostgreSQL tramite JDBC.
Comunicazione	Socket TCP + ObjectInputStream/ObjectOutputStream	Trasporta oggetti serializzabili tra client e server.
Cifratura password	pgcrypto con crypt() e gen_salt('bf')	Evita il salvataggio di password in chiaro.

2.1 Dipendenze Maven principali

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>42.7.4</version>
</dependency>

<dependency>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-controls</artifactId>
  <version>21.0.5</version>
</dependency>
```

Il plugin maven-shade-plugin genera due artefatti separati, uno con main class cinemax.server.ServerCM e uno con main class cinemax.client.AvvioClient. In questo modo server e client possono essere avviati separatamente.

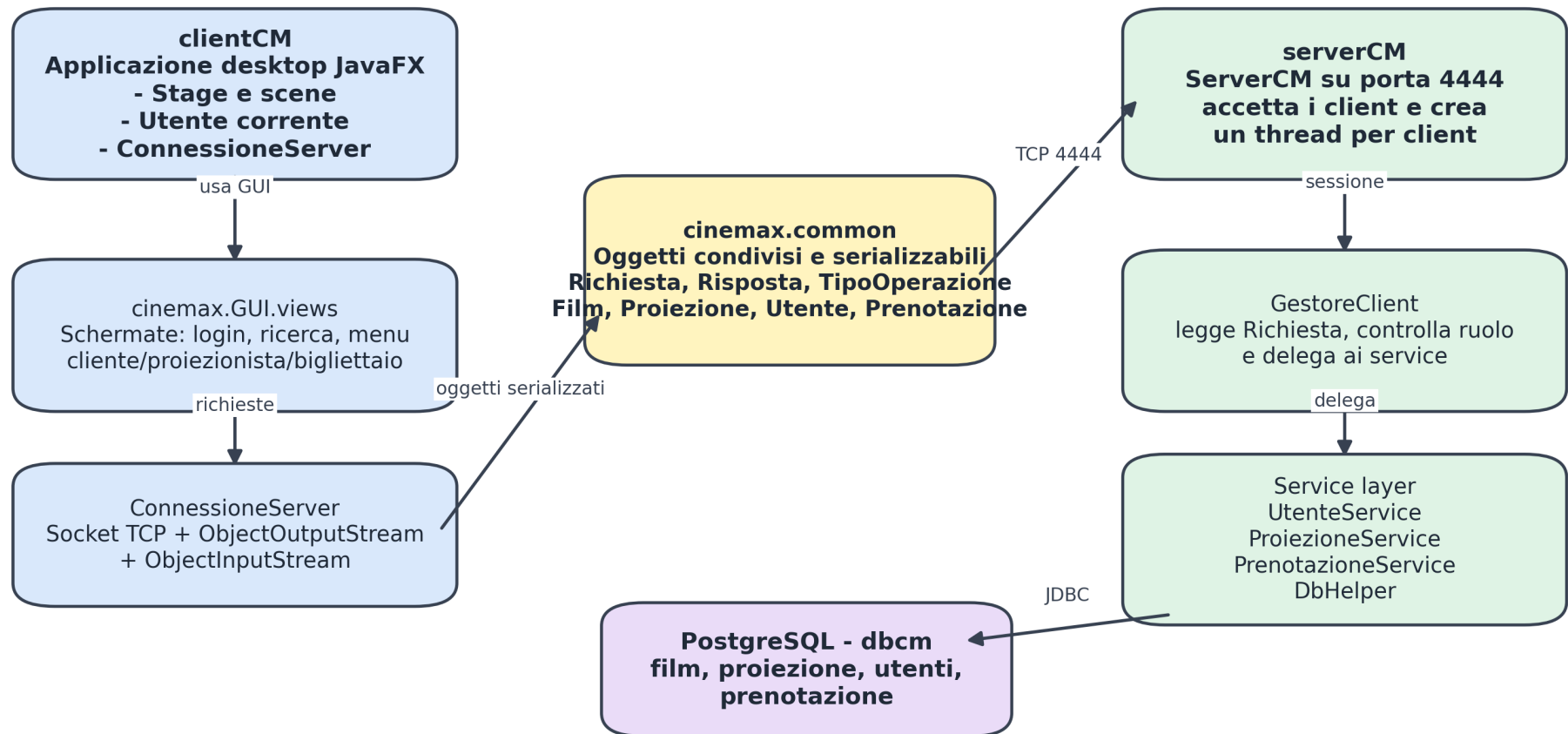
3. Architettura generale client/server

CineMax segue una architettura distribuita a tre livelli logici. Il client JavaFX si occupa dell'interazione con l'utente e non accede direttamente al database. Il server riceve le richieste dal client, controlla autenticazione e autorizzazione, esegue la logica applicativa e comunica con PostgreSQL. Il database conserva in modo persistente tutte le informazioni.

Livello	Componenti	Responsabilità
Presentazione	clientCM, classi in cinemax.GUI.views	Costruzione delle schermate, raccolta input utente, visualizzazione messaggi e tabelle.
Comunicazione	ConnessioneServer, Richiesta, Risposta, TipoOperazione	Invio di oggetti serializzati al server e ricezione delle risposte.
Logica applicativa	serverCM, GestoreClient, service layer	Controllo ruoli, validazione, transazioni, accesso al database.
Persistenza	PostgreSQL dbcm	Memorizzazione e interrogazione di film, proiezioni, utenti e prenotazioni.

3.1 Diagramma architetturale

CineMax - Architettura generale client/server



Idea chiave: client grafico e server sono due JAR separati; il database resta lato server.

4. Struttura del repository e dei package

La struttura del progetto separa codice sorgente, documentazione, file eseguibili e script/dati del database.

Cartella/file	Contenuto
src/	Codice sorgente Java organizzato nei package cinemax.common, cinemax.client, cinemax.GUI.views, cinemax.server e cinemax.server.service.
data/	Script SQL e file CSV usati per creare e popolare il database.
bin/	JAR eseguibili serverCM.jar e clientCM.jar pronti per l'esecuzione.
doc/	Manuali, diagrammi e Javadoc generata.
pom.xml	Configurazione Maven del progetto, dipendenze e plugin di build.
README.txt	Istruzioni sintetiche per creare database, compilare ed eseguire.

4.1 Package Java

Package	Classi principali	Responsabilità
cinemax.common	Film, Proiezione, Prenotazione, Utente, Richiesta, Risposta, TipoOperazione	Oggetti di dominio e oggetti del protocollo condivisi da client e server. Sono serializzabili.
cinemax.client	AvvioClient, ClientCM, ConnessioneServer	Avvio dell'applicazione grafica, gestione stage/scena, connessione TCP al server.
cinemax.GUI.views	ScenaIniziale, ScenaRicercaProiezioni, ScenaDettaglioProiezione, ScenaCreaPrenotazione, menu dei ruoli, ecc.	Schermate JavaFX e gestione eventi utente.
cinemax.server	ServerCM, GestoreClient	Avvio server, accettazione client, ciclo richiesta-risposta e controllo autorizzazioni.
cinemax.server.service	UtenteService, ProiezioneService, PrenotazioneService, DbHelper	Logica applicativa e accesso al database.

4.2 Diagramma dei casi d uso

CineMax - Casi d'uso per ruolo

GUEST	CLIENTE	PROIEZIONISTA	BIGLIETTAIO
Cercare proiezioni	Login / logout	Login / logout	Login / logout
Visualizzare dettagli	Cercare proiezioni	Visualizzare palinsesto	Prenotazioni di oggi
Registrarsi come cliente	Creare prenotazione	Aggiungere proiezione	Cercare prenotazioni
	Visualizzare prenotazioni	Modificare proiezione	
	Modificare prenotazione	Eliminare proiezione	
	Cancellare prenotazione		

Le funzionalità disponibili dipendono dal ruolo. I controlli di autorizzazione reali sono lato server.

5. Protocollo di comunicazione Richiesta/Risposta

Il client non invia comandi SQL al server. Ogni azione dell'utente viene trasformata in un oggetto Richiesta. La richiesta contiene un valore di TipoOperazione e una mappa di parametri. Il server riceve la richiesta, la smista al service corretto e restituisce un oggetto Risposta contenente esito, messaggio e dati.

Classe	Campi/metodi principali	Ruolo tecnico
Richiesta	operazione, parametri; setParametro(), getParametro()	Rappresenta il messaggio inviato dal client al server.
TipoOperazione	LOGIN, CERCA_PROIEZIONE, CREA_PRENOTAZIONE, ecc.	Identifica l'azione richiesta.
Risposta	successo, messaggio, dati; isSuccesso(), getMessaggio(), getDati()	Rappresenta il messaggio di ritorno inviato dal server al client.
Oggetti dominio	Film, Proiezione, Prenotazione, Utente	Possono essere inseriti in dati della risposta per trasportare informazioni complesse.

5.1 Flusso generale di una richiesta

1. La scena JavaFX raccoglie i dati inseriti dall'utente.
2. Il client crea una Richiesta con un TipoOperazione.
3. `ConnessioneServer.invia()` scrive la richiesta sull' `ObjectOutputStream`.
4. `GestoreClient` legge la richiesta dall' `ObjectInputStream`.
5. Il server controlla il ruolo dell'utente autenticato.
6. Il service esegue la logica applicativa e accede al database.
7. Il server restituisce una Risposta al client.
8. La GUI aggiorna la scena o mostra un messaggio all'utente.

6. Modello di dominio e classi comuni

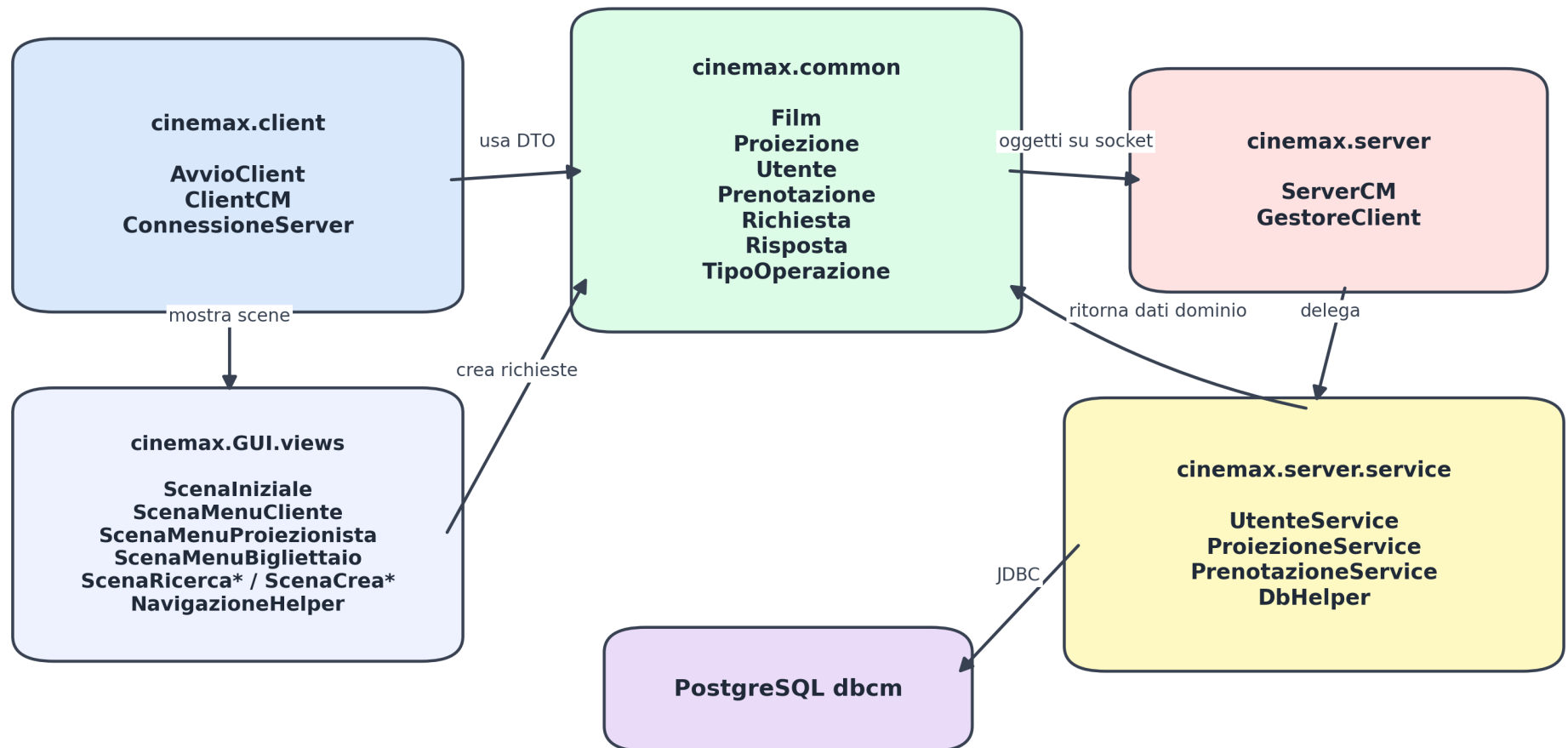
Le classi comuni rappresentano il dominio applicativo e sono condivise tra client e server.

Classe	Campi principali	Descrizione
Film	id, titolo, genere, regista, anno, durataMinuti, etaMinima	Contiene le informazioni anagrafiche del film. Un film può avere più proiezioni.
Proiezione	id, film, dataOra, prezzoBiglietto, postiLiberi	Rappresenta una proiezione concreta di un film in una certa data e ora. La capienza sala è costante a 200 posti.
Prenotazione	codice, username, nomeCliente, cognomeCliente, proiezione, numPosti	Rappresenta la prenotazione di un cliente per una proiezione. Calcola anche il costo totale.
Utente	username, nome, cognome, dataNascita, domicilio, ruolo	Rappresenta utenti di tutti i ruoli: cliente, proiezionista e bigliettaio.

La scelta di rendere serializzabili queste classi permette di inviarle direttamente tra client e server tramite `ObjectInputStream` e `ObjectOutputStream`. Il client riceve oggetti già strutturati e può mostrarli nelle tabelle JavaFX.

6.1 Diagramma UML delle classi principali

CineMax - Diagramma dei package e classi principali



Legenda: le frecce indicano dipendenze principali tra package, non tutte le chiamate interne.

7. Progettazione del database e normalizzazione

Il database PostgreSQL si chiama dbcm. La progettazione separa i dati dei film dalle singole proiezioni per evitare ridondanza: titolo, genere, regista, anno, durata ed età minima appartengono al film; data, ora e prezzo appartengono alla proiezione. Gli utenti sono memorizzati in una unica tabella con attributo ruolo.

Tabella	Chiave primaria	Attributi principali	Vincoli principali
film	id	titolo, genere, regista, anno, durata_minuti, eta_minima	UNIQUE(titolo, anno, regista); controlli su anno, durata ed eta minima.
proiezione	id	film_id, data_ora, prezzo_biglietto	FK verso film; data_ora unica; prezzo non negativo.
utenti	username	nome, cognome, password_cifrata, data_nascita, domicilio, ruolo	CHECK sul ruolo; password cifrata; data_nascita opzionale.
prenotazione	codice	username, proiezione_id, num_posti	FK verso utenti e proiezione; num_posti > 0.

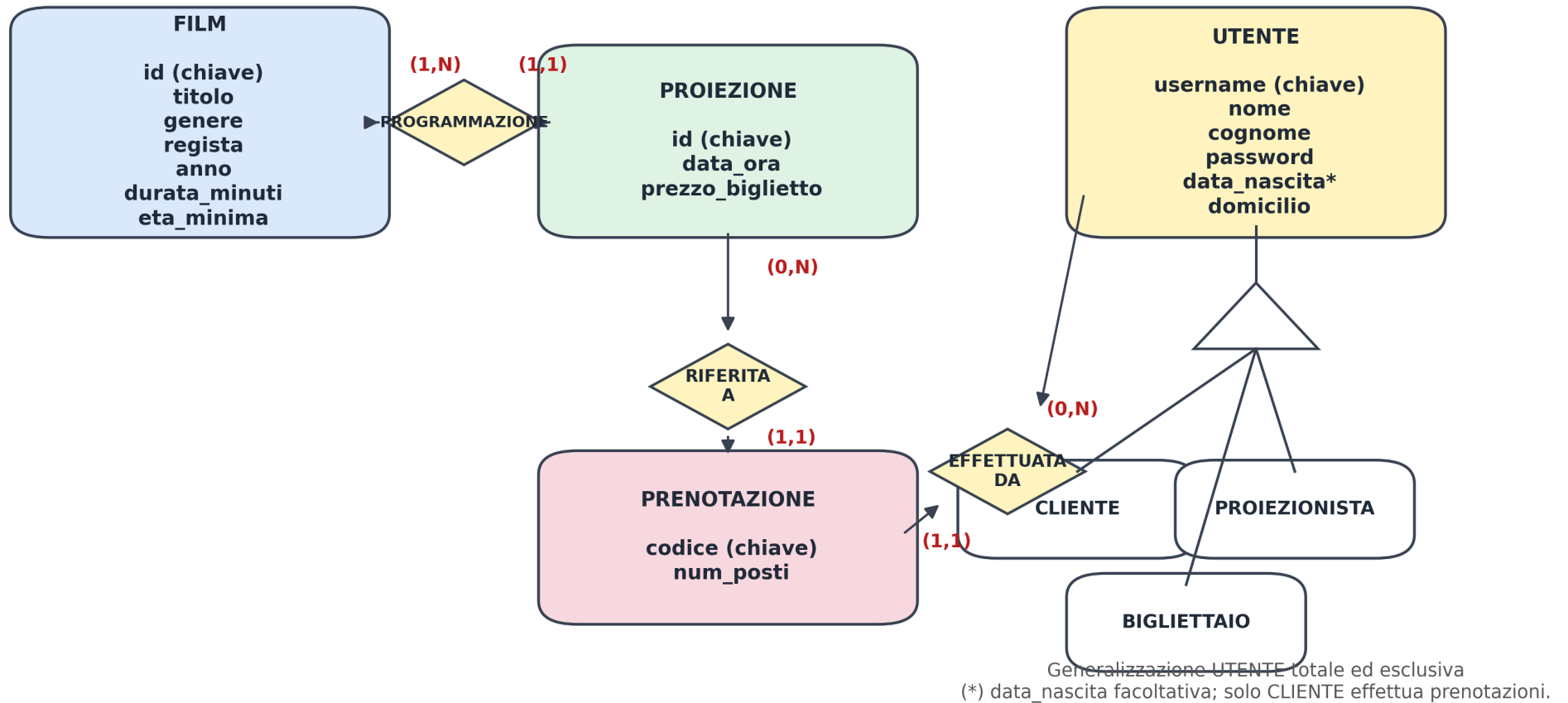
7.1 Normalizzazione

Lo schema è in terza forma normale rispetto alle dipendenze funzionali principali. Gli attributi del film dipendono dall'identificativo del film e non dalla proiezione. Gli attributi della prenotazione dipendono dal codice della prenotazione. Gli attributi dell'utente dipendono dallo username. La generalizzazione Utente -> Cliente/Proiezionista/Bigliettaio è stata implementata con un solo attributo ruolo per semplificare login e autorizzazione.

7.2 Diagramma ER concettuale

7.3

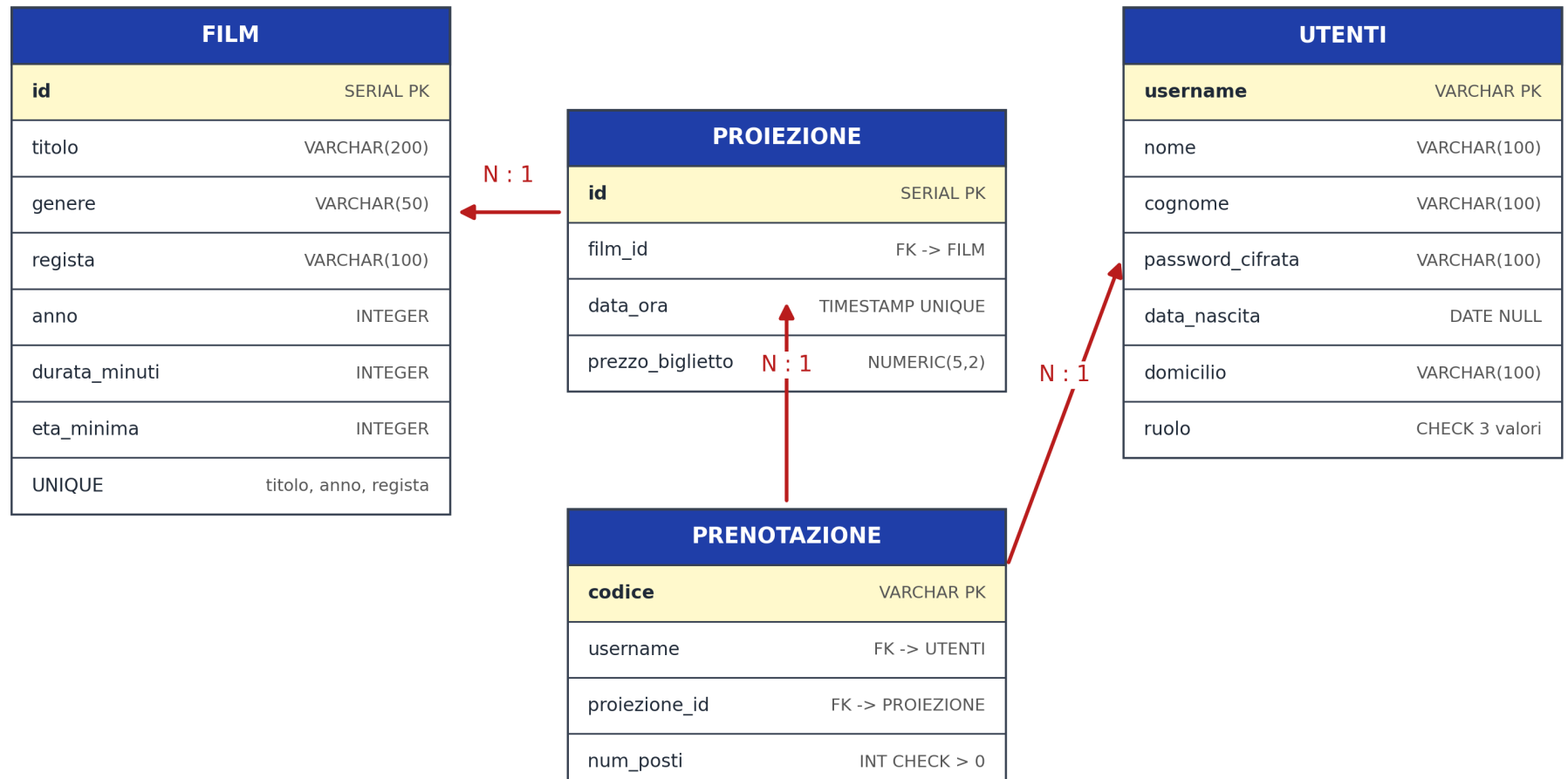
CineMax - Schema ER concettuale semplificato



7.4 Schema relazionale

7.5

CineMax - Schema relazionale ristrutturato (3NF)



Ristrutturazione: la generalizzazione UTENTE è collassata in una sola tabella con attributo ruolo.
Tutte le tabelle sono in 3NF: gli attributi non chiave dipendono dalla chiave della tabella.

8. Modulo serverCM: ServerCM, GestoreClient e service layer

Il server e il componente centrale dell'applicazione. Viene avviato prima del client, chiede all'operatore host, username e password del database, verifica la connessione JDBC e poi resta in ascolto sulla porta TCP 4444.

8.1 ServerCM

ServerCM contiene il metodo main del server. Il suo compito non e eseguire direttamente la logica applicativa, ma preparare l'ambiente del server: costruzione dell'URL JDBC, verifica della connessione al database, apertura del ServerSocket e creazione di un GestoreClient per ogni client accettato.

Metodo/campo	Descrizione
PORTA = 4444	Porta TCP su cui il server riceve connessioni.
NOME_DB = dbcm	Nome del database PostgreSQL usato dal progetto.
main(String[] args)	Legge credenziali DB, verifica la connessione e avvia il ciclo di accettazione client.

8.2 GestoreClient

GestoreClient rappresenta la sessione di un singolo client. Viene eseguito in un thread dedicato e mantiene lo stato dell'utente autenticato. Ogni GestoreClient apre una propria connessione JDBC, istanzia i service e rimane nel ciclo lettura richiesta - elaborazione - invio risposta.

Metodo/campo	Descrizione
run()	Apri connessione JDBC, crea stream oggetti, legge richieste e restituisce risposte fino alla disconnessione.
elabora(Richiesta)	Smista la richiesta in base a TipoOperazione e controlla le autorizzazioni lato server.
haRuolo(String ruolo)	Verifica che l'utente autenticato abbia il ruolo richiesto.
chiudiRisorse()	Chiude connessione al database e socket del client.
utenteAutenticato	Memorizza l'utente loggato nella sessione corrente.

8.3 Service layer

Il service layer contiene la logica applicativa vera e propria. Questa scelta evita di avere codice SQL o regole di business nelle classi GUI. Ogni service riceve una connessione JDBC e restituisce sempre oggetti Risposta.

Service	Responsabilità
UtenteService	Login e registrazione dei clienti; cifratura e verifica password tramite pgcrypto.
ProiezioneService	Ricerca e visualizzazione proiezioni; aggiunta, modifica, eliminazione e palinsesto per il proiezionista.
PrenotazioneService	Creazione, visualizzazione, modifica, eliminazione e ricerca prenotazioni; transazioni per le operazioni critiche.
DbHelper	Frammenti SQL condivisi e conversione da ResultSet a oggetti Proiezione/Prenotazione.

9. Modulo clientCM: applicazione JavaFX e scene principali

Il clientCM è una applicazione desktop JavaFX. Non contiene la logica di business principale: raccoglie input, crea richieste, mostra risposte e cambia scena in base al ruolo dell'utente.

Classe/scena	Funzione
AvvioClient	Launcher del client; richiama ClientCM.main() per avviare JavaFX.
ClientCM	Estende Application, gestisce Stage principale, connessione server e utente corrente.
ConnessioneServer	Apri socket TCP e invia richieste al server in modo sincronizzato.
ScenaConnessione	Permette di indicare host e porta se la connessione automatica non riesce.
ScenaIniziale	Login, registrazione e accesso guest.
ScenaRicercaProiezioni	Ricerca delle proiezioni con filtri.
ScenaDettaglioProiezione	Mostra dettagli film/proiezione e permette prenotazione ai clienti.
ScenaCreaPrenotazione	Form per scegliere il numero di posti e inviare CREA_PRENOTAZIONE.
ScenaMenuCliente	Prenotazioni attive del cliente, modifica e cancellazione.
ScenaMenuProiezionista	Palinsesto, aggiunta, modifica ed eliminazione proiezioni.
ScenaMenuBigliettaio	Prenotazioni di oggi e ricerca prenotazioni.
NavigazioneHelper	Metodi statici per tornare al menu corretto e creare bottone Home.

10. Gestione delle prenotazioni, posti disponibili e concorrenza

La gestione dei posti è la parte più critica del progetto. La sala ha 200 posti e più client possono prenotare contemporaneamente la stessa proiezione. Per evitare overbooking, i posti liberi non vengono memorizzati come valore fisso nella tabella proiezione, ma vengono calcolati ogni volta partendo dalla tabella prenotazione.

```
posti_liberi = Proiezione.CAPIENZA_SALA - SUM(num_posti delle prenotazioni della stessa proiezione)
```

È importante distinguere tra film e proiezione: lo stesso film può comparire in più proiezioni, ma la capienza di 200 posti si applica alla singola proiezione identificata da proiezione.id, non al titolo del film.

10.1 Scenario di concorrenza

```
Client A vede 200 posti liberi e prova a prenotare 150 posti.  
Client B vede 200 posti liberi e prova quasi nello stesso momento a prenotare 100 posti.  
Senza transazione corretta il totale diventerebbe 250 posti.  
Con SELECT FOR UPDATE il secondo client attende il commit del primo e poi ricalcola i posti liberi.
```

10.2 Strategia lato server

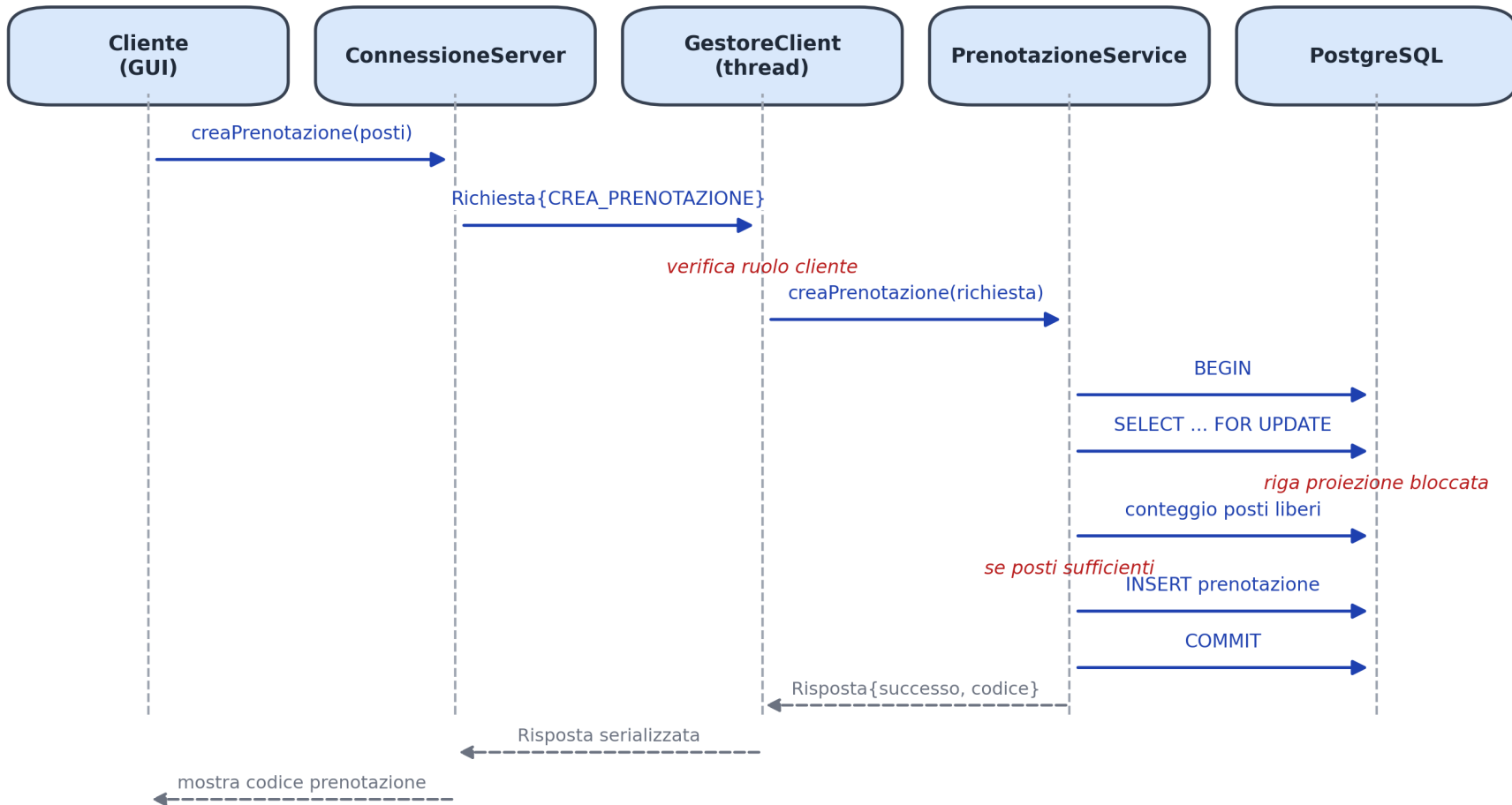
La correzione deve stare lato server. La GUI può mostrare un valore non aggiornato se due client sono aperti contemporaneamente, ma al momento della prenotazione il server deve ricalcolare i posti reali e rifiutare la richiesta se non sono sufficienti. La versione finale di creaPrenotazione deve seguire questi passi:

```
1. conn.setAutoCommit(false)
2. SELECT id, data_ora FROM proiezione WHERE id = ? FOR UPDATE
3. Verifica che la proiezione esista e non sia passata
4. Calcola prenotati = SUM(num_posti) WHERE proiezione_id = ?
5. Calcola liberi = 200 - prenotati
6. Se numPosti > liberi: rollback e risposta di errore
7. Altrimenti INSERT nella tabella prenotazione
8. commit e risposta con codice prenotazione
```

Il lock su proiezione serializza le prenotazioni concorrenti della stessa proiezione. Il secondo client rimane in attesa finché la transazione del primo client termina; solo dopo legge il conteggio aggiornato delle prenotazioni.

10.3 Diagramma di sequenza della prenotazione

CineMax - Sequenza creazione prenotazione con concorrenza



11. Sicurezza, autorizzazione e gestione degli errori

11.1 Password e autenticazione

Le password non sono memorizzate in chiaro. In fase di registrazione viene usata la funzione `crypt(password, gen_salt('bf'))` di `pgcrypto`. In fase di login la password inserita viene confrontata nel database con `password_cifrata = crypt(?, password_cifrata)`.

Aspetto	Soluzione
Password	Hash BCrypt tramite <code>pgcrypto</code> ; nessuna password in chiaro nella tabella utenti.
SQL injection	Uso di <code>PreparedStatement</code> con parametri, evitando concatenazione diretta dell'input utente.
Autorizzazione	Controllo lato server nel metodo <code>GestoreClient.elabora()</code> .
Ruoli	cliente, proiezionista, bigliettotaio, vincolati con <code>CHECK</code> nel database.
Sessione	utenteAutenticato e mantenuto nel <code>GestoreClient</code> del singolo client.
Errori	Ogni service restituisce <code>Risposta(false, messaggio, null)</code> in caso di errore.

11.2 Autorizzazione lato server

L'interfaccia grafica nasconde le funzioni non disponibili per un ruolo, ma questo non è sufficiente per la sicurezza. Il controllo definitivo avviene lato server: prima di eseguire operazioni come `CREA_PRENOTAZIONE`, `AGGIUNGI_PROIEZIONE` o `CERCA_PRENOTAZIONE`, `GestoreClient` verifica il ruolo dell'utente autenticato.

12. Descrizione completa delle funzioni/metodi principali

Le tabelle seguenti riassumono le classi e i metodi principali, indicando responsabilità e note tecniche.

12.1 Package `cinemax.common`

Classe	Metodo/funzione	Spiegazione
Film	<code>Film(...)</code>	Costruttore con dati anagrafici del film.
Film	<code>getId()</code> , <code>getTitolo()</code> , <code>getGenere()</code> , <code>getRegista()</code> , <code>getAnno()</code> , <code>getDurataMinuti()</code> , <code>getEtaMinima()</code>	Getter usati da GUI, service e tabelle JavaFX.
Film	<code>toString()</code>	Rappresentazione testuale del film.
Proiezione	<code>Proiezione(...)</code>	Costruisce una proiezione con film, data/ora, prezzo e posti liberi.
Proiezione	<code>CAPENZA_SALA</code>	Costante di dominio pari a 200.
Proiezione	<code>getId()</code> , <code>getFilm()</code> , <code>getDataOra()</code> , <code>getPrezzoBiglietto()</code> , <code>getPostiLiberi()</code>	Accesso ai dati della proiezione.
Prenotazione	<code>Prenotazione(...)</code>	Costruisce una prenotazione con codice, cliente, proiezione e numero posti.
Prenotazione	<code>getCostoTotale()</code>	Calcola <code>prezzoBiglietto * numPosti</code> .
Utente	<code>Utente(...)</code>	Costruisce un utente senza password, per non trasportare credenziali al client.
Utente	<code>RUOLO_CLIENTE</code> , <code>RUOLO_PROIEZIONISTA</code> , <code>RUOLO_BIGLIETTAIO</code>	Costanti per confrontare i ruoli in modo uniforme.
Richiesta	<code>Richiesta(TipoOperazione)</code>	Crea una richiesta applicativa.
Richiesta	<code>setParametro()</code> , <code>getParametro()</code>	Gestisce la mappa dei parametri della richiesta.
Risposta	<code>Risposta(successo, messaggio, dati)</code>	Crea la risposta del server.
Risposta	<code>isSuccesso()</code> , <code>getMessaggio()</code> , <code>getDati()</code>	Permette alla GUI di decidere cosa mostrare.
TipoOperazione	valori enum	Elenca tutte le operazioni ammesse dal protocollo.

12.2 Package cinemax.client

Classe	Metodo/funzione	Spiegazione
AvvioClient	main(String[] args)	Punto di ingresso del JAR client.
ClientCM	start(Stage)	Avvia JavaFX, imposta stage principale e prova la connessione automatica.
ClientCM	mostraScena(Scene)	Sostituisce la scena attualmente visibile.
ClientCM	getConnessione()/setConnessione()	Gestisce la connessione TCP condivisa dal client.
ClientCM	getUtenteCorrente()/setUtenteCorrente()	Mantiene l'utente autenticato lato client.
ClientCM	stop()	Chiude la connessione quando l'applicazione termina.
ConnessioneServer	ConnessioneServer(host, porta)	Apri socket e stream oggetti verso il server.
ConnessioneServer	invia(Richiesta)	Metodo synchronized: invia una richiesta e attende una risposta.
ConnessioneServer	chiudi()	Chiude il socket del client.

12.3 Package cinemax.server

Classe	Metodo/funzione	Spiegazione
ServerCM	main(String[] args)	Avvia il server, verifica il database e accetta connessioni sulla porta 4444.
GestoreClient	run()	Gestisce il ciclo di vita di una sessione client in un thread dedicato.
GestoreClient	elabora(Richiesta)	Smista le operazioni e controlla autorizzazioni in base al ruolo.
GestoreClient	haRuolo(String)	Controlla che l'utente autenticato abbia il ruolo richiesto.
GestoreClient	chiudiRisorse()	Chiude connessione JDBC e socket.

12.4 Package cinemax.server.service

Classe	Metodo/funzione	Spiegazione
UtenteService	login(Richiesta)	Verifica username e password cifrata, restituisce Utente senza password.
UtenteService	registraCliente(Richiesta)	Inserisce un cliente con password cifrata e gestisce username duplicati.
ProiezioneService	cercaProiezione(Richiesta)	Ricerca proiezioni con filtri opzionali: titolo, genere, data, prezzo.
ProiezioneService	visualizzaProiezione(Richiesta)	Restituisce dettaglio di una proiezione con conteggio aggiornato posti liberi.
ProiezioneService	aggiungiProiezione(Richiesta)	Crea film se necessario, controlla sovrapposizioni e inserisce la proiezione.
ProiezioneService	modificaProiezione(Richiesta)	Modifica data/ora se non ci sono prenotazioni e non c'è sovrapposizione.
ProiezioneService	eliminaProiezione(Richiesta)	Elimina solo se non esistono prenotazioni collegate.
ProiezioneService	visualizzaPalinsesto(Richiesta)	Mostra proiezioni future o storiche.
PrenotazioneService	creaPrenotazione(Richiesta)	Operazione critica: transazione, lock FOR UPDATE, controllo posti e inserimento prenotazione.
PrenotazioneService	visualizzaPrenotazioni(Richiesta)	Mostra le prenotazioni future del cliente.
PrenotazioneService	modificaPrenotazione(Richiesta)	Sposta la prenotazione a una nuova proiezione dello stesso film, controllando data e posti.
PrenotazioneService	eliminaPrenotazione(Richiesta)	Cancella prenotazioni future.
PrenotazioneService	cercaPrenotazione(Richiesta)	Ricerca per biglietti con filtri codice, nome, cognome, titolo e data.
PrenotazioneService	prenotazioniOggi()	Restituisce le prenotazioni della data corrente.
DbHelper	SQL_PROIEZIONE	Query base che calcola posti_liberi con subquery su prenotazione.
DbHelper	SQL_PRENOTAZIONE	Query base per prenotazioni con dati cliente, film e proiezione.
DbHelper	proiezioneFromRs(), prenotazioneFromRs()	Convertono ResultSet in oggetti di dominio.

12.5 Package cinemax.GUI.views

Classe/scena	Metodi principali	Spiegazione
ScenaIniziale	costruisci(), eseguiLogin()	Schermata di accesso, registrazione e guest; invia LOGIN al server.
ScenaConnessione	costruisci()	Permette connessione manuale a host/porta.
ScenaRegistrazione	costruisci()	Form di registrazione cliente; invia REGISTRA_CLIENTE.
ScenaRicercaProiezioni	costruisci()	Filtri di ricerca e tabella risultati; invia CERCA_PROIEZIONE.
ScenaDettaglioProiezione	costruisci(), aggiungiRiga()	Mostra dettagli proiezione e abilita prenotazione per cliente.
ScenaCreaPrenotazione	costruisci(), aggiungiRiga()	Invia CREA_PRENOTAZIONE con numero posti scelto.
ScenaMenuCliente	costruisci(), mostraDialogModifica(), mostraDialogCancella(), eseguiLogout()	Menu cliente e gestione prenotazioni.
ScenaMenuProiezionista	costruisci(), mostraDialogModifica(), mostraDialogElimina(), eseguiLogout()	Menu palinsesto proiezionista.
ScenaAggiungiProiezione	costruisci()	Form per inserire nuova proiezione e dati film.
ScenaMenuBigliettaio	costruisci(), costruisciTabella(), eseguiLogout()	Menu bigliettotaio con prenotazioni di oggi.
ScenaRicercaPrenotazioni	costruisciPannello(), costruisci()	Pannello/tab per ricerca prenotazioni.
NavigazioneHelper	tornaAlMenu(), creaBottoneHome()	Navigazione comune per tornare al menu corretto.

13. Operazioni del protocollo applicativo

Operazione	Ruolo richiesto	Service chiamato	Descrizione
LOGIN	Nessuno	UtenteService.login	Autenticazione utente registrato.
LOGOUT	Utente loggato	GestoreClient	Azzera l'utente autenticato nella sessione.
REGISTRA_CLIENTE	Nessuno	UtenteService.registraCliente	Crea nuovo cliente.
CERCA_PROIEZIONE	Nessuno/guest	ProiezioneService.cercaProiezione	Ricerca proiezioni con filtri.
VISUALIZZA_PROIEZIONE	Nessuno/guest	ProiezioneService.visualizzaProiezione	Dettaglio proiezione e posti aggiornati.
CREA_PRENOTAZIONE	cliente	PrenotazioneService.creaPrenotazione	Crea prenotazione se posti sufficienti.
VISUALIZZA_PRENOTAZIONI	cliente	PrenotazioneService.visualizzaPrenotazioni	Elenco prenotazioni future del cliente.
MODIFICA_PRENOTAZIONE	cliente	PrenotazioneService.modificaPrenotazione	Sposta prenotazione a nuova data dello stesso film.
ELIMINA_PRENOTAZIONE	cliente	PrenotazioneService.eliminaPrenotazione	Cancella prenotazione futura.
CERCA_PRENOTAZIONE	bigliettaio	PrenotazioneService.cercaPrenotazione	Ricerca prenotazioni con filtri.
PRENOTAZIONI_OGGI	bigliettaio	PrenotazioneService.prenotazioniOggi	Mostra prenotazioni della giornata.
AGGIUNGI_PROIEZIONE	proiezionista	ProiezioneService.aggiungiProiezione	Inserisce nuova proiezione se non si sovrappone.
MODIFICA_PROIEZIONE	proiezionista	ProiezioneService.modificaProiezione	Modifica data/ora se non esistono prenotazioni.
ELIMINA_PROIEZIONE	proiezionista	ProiezioneService.eliminaProiezione	Elimina proiezione senza prenotazioni.
VISUALIZZA_PALINSESTO	proiezionista	ProiezioneService.visualizzaPalinsesto	Mostra proiezioni future o storiche.

14. Build, esecuzione e distribuzione

14.1 Creazione database

```
psql -U postgres -h localhost -c "CREATE DATABASE dbcm;"
cd data
psql -U postgres -h localhost -d dbcm -f crea_database.sql
```

Lo script crea le tabelle, abilita pgcrypto, inserisce gli utenti predefiniti e importa il palinsesto dal CSV. Se l'import delle proiezioni è separato, va eseguito anche lo script dedicato presente nella cartella data.

14.2 Compilazione e JAR

```
mvn clean package
mvn javadoc:javadoc
```

Dopo la compilazione, i JAR finali sono disponibili in target/ e possono essere copiati nella cartella bin/.

14.3 Avvio server e client

```
java -jar bin/serverCM.jar
# inserire host DB, username e password PostgreSQL

java -jar bin/clientCM.jar
```

Il server deve essere avviato prima del client. Quando il server stampa "Server in ascolto sulla porta 4444", il client può collegarsi a localhost:4444 oppure a un host diverso indicato nella schermata di connessione.

15. Collaudo funzionale e casi di test

Caso di test	Passi	Risultato atteso
Avvio server	Avviare serverCM.jar e inserire credenziali PostgreSQL corrette.	Messaggio di connessione al database verificata e server in ascolto sulla porta 4444.
Connessione client	Avviare clientCM.jar con server acceso.	Il client si collega e mostra la schermata iniziale.
Login cliente	Inserire test/test o cliente registrato.	Apertura menu cliente.
Login errato	Inserire username o password sbagliati.	Messaggio Username o password errati.
Registrazione cliente	Compilare form registrazione con username nuovo.	Cliente inserito e login possibile.
Username duplicato	Registrare un cliente con username già presente.	Messaggio di username già in uso.
Ricerca guest	Cercare titolo parziale senza login.	Lista proiezioni e dettaglio consultabile, ma nessuna prenotazione.
Prenotazione valida	Cliente sceglie proiezione futura con posti liberi e prenota.	Codice prenotazione generato.
Prenotazione oltre disponibilità	Provare a prenotare più posti di quelli liberi.	Risposta di errore Posti insufficienti.
Prenotazione proiezione passata	Provare a prenotare data/ora scaduta.	Risposta di errore: proiezione passata non prenotabile.
Concorrenza	Due client prenotano contemporaneamente la stessa proiezione.	Il totale non supera 200; il secondo client riceve errore se i posti non bastano.
Proiezionista overlap	Aggiungere proiezione che si sovrappone a una già esistente.	Operazione rifiutata.
Eliminazione proiezione con prenotazioni	Eliminare una proiezione con prenotazioni presenti.	Operazione rifiutata.
Bigliettaio	Login bigliettotaio e ricerca per codice/nome/titolo/data.	Tabella prenotazioni coerente con filtri.

16. Complessità, prestazioni e scelte progettuali

Le prestazioni del sistema dipendono principalmente dal database, poiché le operazioni più costose sono ricerche, join e conteggi. Il codice Java delega i filtri a PostgreSQL, evitando di caricare tutti i dati in memoria lato client.

Operazione	Complessità/considerazioni
Getter e oggetti dominio	Tempo costante $O(1)$.
cercaProiezione	Dipende dal numero di proiezioni che soddisfano i filtri; il database usa indice su data_ora.
visualizzaProiezione	Ricerca per id proiezione; include subquery per sommare i posti prenotati.
creaPrenotazione	Transazione breve: lock della proiezione, somma dei posti prenotati e insert. La serializzazione avviene solo sulla proiezione interessata.
cercaPrenotazione	Dipende dai filtri e dal numero di prenotazioni; usa join tra prenotazione, utenti, proiezione e film.
ObjectOutputStream.reset()	Evita che il client riceva oggetti vecchi dalla cache di serializzazione.

16.1 Scelte progettuali principali

Scelta	Motivazione
Thread per client	Soluzione semplice e adeguata al progetto didattico; consente più sessioni contemporanee.
Service layer	Mantiene separata la logica applicativa dalla GUI e dal ciclo socket.
PreparedStatement	Rende più sicuro l'accesso al database e semplifica i parametri.
Ruolo nella tabella utenti	Evita tre tabelle separate per utenti con pochi attributi diversi.
Calcolo dinamico posti liberi	Evita inconsistenze tra valore salvato e prenotazioni effettive.
SELECT FOR UPDATE	Previene overbooking in presenza di richieste concorrenti.

17. Limiti, assunzioni e possibili estensioni

Aspetto	Descrizione
Cinema monosala	La capienza è unica e fissata a 200 posti. Non sono gestite sale multiple.
Aggiornamento GUI	Le finestre già aperte non ricevono aggiornamenti real-time. I posti vengono però ricalcolati dal server al momento della prenotazione.
Rete	La comunicazione TCP non usa cifratura TLS; per un ambiente reale sarebbe opportuno aggiungerla.
Password database	Le credenziali PostgreSQL sono inserite all'avvio del server e non salvate nel codice.
Pagamento	La prenotazione non gestisce pagamento reale, solo costo totale.
Persistenza log	Gli errori sono mostrati in console o nella risposta; si potrebbe aggiungere logging strutturato.
Estensioni future	Sale multiple, report incassi, aggiornamento real-time, gestione posti numerati, pannello amministratore.

Appendice A. Comandi utili

```
# Creazione database
psql -U postgres -h localhost -c "CREATE DATABASE dbcm;"

# Esecuzione script da cartella data
cd data
psql -U postgres -h localhost -d dbcm -f crea_database.sql

# Compilazione
mvn clean package

# Javadoc
mvn javadoc:javadoc

# Avvio server
java -jar bin/serverCM.jar

# Avvio client
java -jar bin/clientCM.jar

# Controllo prenotazioni per proiezione
SELECT p.id, f.titolo, p.data_ora,
       COALESCE(SUM(pr.num_posti), 0) AS posti_prenotati,
       200 - COALESCE(SUM(pr.num_posti), 0) AS posti_liberi
FROM proiezione p
JOIN film f ON p.film_id = f.id
LEFT JOIN prenotazione pr ON pr.proiezione_id = p.id
GROUP BY p.id, f.titolo, p.data_ora
ORDER BY p.data_ora;
```

Appendice B. Riferimenti interni al progetto

File principali da consultare insieme al manuale tecnico:

File	Contenuto
README.txt	Istruzioni rapide per database, compilazione, esecuzione e credenziali di prova.
pom.xml	Dipendenze JavaFX/PostgreSQL e plugin Maven per JAR e Javadoc.
crea_database.sql	Schema SQL, estensione pgcrypto, utenti predefiniti e import dati.
ServerCM.java	Main server, porta 4444 e accettazione client.
GestoreClient.java	Sessione client, stream oggetti, autorizzazione e smistamento richieste.
UtenteService.java	Login e registrazione.
ProiezioneService.java	Gestione proiezioni e palinsesto.
PrenotazioneService.java	Gestione prenotazioni, transazioni e concorrenza.
DbHelper.java	Query comuni e mapping ResultSet-oggetti.
classi in cinemax.GUI.views	Scene JavaFX del client.